

AI Usage Log

Sprint 1 | SOEN 341 Software Process, Fall 2026
Project: CareerConnect, Job Search and Application Tracking Platform

Team member	Sayf
Repository path	AI_Log / Sayf /
Sprint	Sprint 1
Entries logged	5 (Task 1.1, 1.2, 1.3, 1.4, 1.5)

Highlighted text is draft wording for validation, decision, or reflection that the team member should confirm or edit before submission.

Summary of Entries

ID	Task	Purpose	Decision	Evidence
1.1	Generate 10 user stories	Brainstorming, requirements	Accepted (AI batch)	Appendix A
1.2	Test database connection and merge User model	Debugging, Git workflow	Modified before use	Chat link, Appendix B
1.3	Format Sprint 1 planning document	Documentation	Accepted with edits	Appendix C
1.4	Format this AI usage log	Documentation	Accepted with edits	Appendix D
1.5	Debug registration and login, review README and issues	Debugging, documentation review	Modified before use	Chat link, Appendix E

Task 1.1: Generate 10 User Stories

Purpose of AI Use	Brainstorming and requirements elicitation. The assignment requires one log entry in which an AI assistant is prompted to generate 10 user stories for the project.
AI tool	Claude (Anthropic)
Chat Link or Prompt/Response	Full prompt and response included in Appendix A.
AI-Suggested Content	• Job seeker (5): upload and update resume; track application status (Applied, Interview, Offered, Rejected); deadline notifications; application dashboard; AI-generated resume feedback. • Recruiter (5): create and publish postings; edit or close postings; view applicants per posting; update applicant status; search and filter candidate profiles by skills or experience.
Validation	Compared the suggested stories against the CareerConnect feature list in the assignment brief and against the team's existing stories to check for gaps and overlap.
Decision	Accepted as the AI-generated batch. Kept in a section separate from the team-generated user stories and features, as required.
Reflection	The AI covered recruiter-side and tracking areas that were missing from our first draft. The stories are fairly generic, so the team wrote additional original stories separately rather than relying on this list.
Responsible Person	Sayf

Task 1.2: Test Database Connection and Merge User Model

Purpose of AI Use	Debugging and Git workflow guidance. Verify that the SQLAlchemy connection to the Neon cloud PostgreSQL database works, then merge the backend foundation (connection, Base, User model) into main.
AI tool	ChatGPT (OpenAI), per the shared link
Chat Link or Prompt/Response	Chat link: https://chatgpt.com/share/6ab9b5d9-0fe8-83ea-abc9-64c0d3adb21c Key prompts and responses are also excerpted in Appendix B.
AI-Suggested Content	• A temporary test_db.py script that imports the engine, Base, and User model, then calls Base.metadata.create_all() to build the users table. • After an ImportError: switch the imports to the App capitalization and confirm connection.py was saved. • After ModuleNotFoundError: run pip install "psycopg[binary]" and re-freeze requirements.txt. • Git steps: delete the temporary script, check git status so .env and .venv/ are not tracked, commit, push feature/setup-database, open a pull request, and merge.
Validation	• Ran the test script locally inside the virtual environment. • Run 1 failed with an ImportError. Run 2 failed with ModuleNotFoundError: No module named 'psycopg'. • After installing the driver, the script ran successfully and the database connection and table creation worked.

Decision	Modified before use. The test script was corrected after the first error, and the psycopg driver was added to the environment. The temporary test script was not committed.
Reflection	The AI's first fix (import capitalization) did not resolve the real problem. The actual cause, a missing PostgreSQL driver, only appeared after the next run. We learned to read the full traceback and to verify AI diagnoses by running them instead of assuming they are correct.
Responsible Person	Sayf

Task 1.3: Format the Sprint 1 Planning Document

Purpose of AI Use	Documentation. Turn the team's rough sprint planning notes into a properly formatted Sprint 1 plan.
AI tool	Claude (Anthropic)
Chat Link or Prompt/Response	Full prompt (the team's notes) and the AI's reply included in Appendix C. The formatted plan itself is a separate document.
AI-Suggested Content	- A cleaned-up, structured Sprint 1 plan built from the team's notes. - The team-capacity calculation the notes had left open: 6 people x 4 hrs/week x 2 weeks = 48 total hours, with about 10 hours reserved for ramp-up and about 38 hours for feature work and documentation.
Validation	Capacity arithmetic re-checked: $6 \times 4 \times 2 = 48$; 20% of 48 = 9.6, rounded to 10; $48 - 10 = 38$. Formatted plan reviewed against the team's original notes to confirm no decisions were changed.
Decision	Accepted with edits. All planning decisions (GitHub Flow, one-approval pull requests, Definition of Ready and Done, risks, Fibonacci story points, two-feature demo scope) came from the team's own notes; the AI supplied structure and the capacity math.
Reflection	Useful for saving formatting time and for finishing the capacity calculation. The content and decisions remain the team's own.
Responsible Person	Sayf

Task 1.4: Format This AI Usage Log

Purpose of AI Use	Documentation. Turn the team member's raw notes and copied conversations (user stories, database testing, sprint planning) into a log in the format required by the assignment, and export it as a PDF.
AI tool	Claude (Anthropic)
Chat Link or Prompt/Response	Prompts and the AI's replies included in Appendix D. The uploaded inputs were the assignment brief and the team member's raw AI notes.
AI-Suggested Content	• A structured PDF with a summary table and one entry per AI use, using the required fields (Task, Purpose, Chat Link or Prompt/Response, AI-Suggested Content, Validation, Decision, Reflection, Responsible Person), plus appendices with the full prompts and responses. • Draft wording for validation, decision, and reflection fields, highlighted so the team member can confirm or change it. • Flags for the team member: the Task 1.1 prompt included five existing stories rather than a clean request for 10; the first AI fix in Task 1.2 was not the real cause of the error; tool names and full name needed confirmation.
Validation	Checked the entry fields against the log format and the example entry in the assignment brief, and confirmed that the prompts, code, and user stories in the appendices match the original conversations. Draft fields were reviewed and corrected by the team member.
Decision	Accepted with edits. The team member confirmed or replaced the highlighted draft text; the facts about what was prompted and what the AI produced come from the original conversations.
Reflection	Saved time on formatting and made sure no required field was missed. The AI could not know our real validation steps, so those had to come from the team member. It is important not to let AI invent validation or reflection content.
Responsible Person	Sayf

Task 1.5: Debug Registration and Login, Review README and Issues

Purpose of AI Use	Debugging and documentation review. Fix the backend errors that blocked registration and login on the team member's local machine (an outdated database table, a CORS rejection, a missing JWT secret key), then check the team README and the GitHub issue list against the assignment requirements. The team member also instructed the AI, twice, to add this conversation to the log; the first instruction also asked the AI to record that the team member gave it.
AI tool	Claude (Anthropic)
Chat Link or Prompt/Response	Chat link: https://claude.ai/share/74720ca5-a9c7-4db0-8244-e7ebe5e4a248 Key prompts and responses are also excerpted in Appendix E.

AI-Suggested Content	• Registration returned a 500 error. Diagnosed as the users table in Neon being out of date: the User model has a <code>full_name</code> column but the table was created before it existed. Suggested a small Python script (<code>reset_db.py</code>) that runs <code>drop_all</code> then <code>create_all</code>, with Alembic named as the long-term fix. • Registration then returned 400 on the OPTIONS request. Diagnosed as CORS: the frontend ran on port 5175 while the backend only allowed 5173. Suggested adding a <code>CORS_ORIGINS</code> line to <code>.env</code>. • Login returned a 500 error. Diagnosed as <code>JWT_SECRET_KEY</code> missing from <code>.env</code>, so the token could not be signed. The AI generated a random 64-character key for local development. • README review. Listed sections the assignment requires but the README lacked (problem, solution, features, GitHub link), a duplicated team member line, and missing <code>.env</code> variables in the setup steps, and drafted replacement sections. • Issue list review. The team member said the README features are the user stories, already created as GitHub issues, and pasted the list. The AI turned the issues into the README Features section and filled in the repository link. It also flagged duplicate task issues (#10/#12 and #11/#13), missing labels to separate AI-generated from team-generated stories, inconsistent label capitalization, and that story #26 had been labeled "AI suggestion 3" in the Task 1.1 prompt. • Documentation. At the team member's request, wrote this Task 1.5 entry and Appendix E from the conversation, then extended them when the team member asked for the later messages to be added too. Both instructions, and the instruction to state that the team member gave them, came from the team member (see Appendix E, Parts 6 and 8).
Validation	Ran the reset script, restarted the backend and frontend, and confirmed in the server log that registration returned 201 Created. Login was re-tested after adding <code>JWT_SECRET_KEY</code>: team member to confirm the result. README suggestions were compared with the required README list in the assignment brief. The issue list was checked against the AI-generated stories in Appendix A. Team member to confirm whether the duplicate issues were closed and the labels added.
Decision	Modified before use. The AI guessed a wrong file name in the reset script, so the import was corrected to the project's real database file. The CORS and secret-key values were set in the team member's own local <code>.env</code>. README and issue-list suggestions are for the team to review.
Reflection	The AI's diagnoses matched the error messages, but it had to guess file names until the real code was pasted. The fixes lived in one machine's <code>.env</code>, so teammates need the same settings. The secret key was generated inside a chat, so it is for local development only and is redacted from this log.
Responsible Person	Sayf

Appendix A: Task 1.1 Prompt and Response

Prompt (as submitted)

Here are my 5 user stories:
The five:
AI suggestion 3: the job seeker can provide the career goals and their CV to be able to match the user with the most appropriate jobs.
filter job postings (by location, title, or category) so that I can quickly find roles relevant to me.
As a job seeker, I want to save jobs to a favourites list so that I can revisit postings I'm interested in later.
As a job seeker, I want to create an account and log in to access my profile and View application history.
recruiters can view my qualifications when I apply to jobs.
Create me 10 more users stories

AI response

Here are 10 additional user stories for CareerConnect, covering areas your current five don't touch yet: resume management, recruiter-side features, application tracking, and notifications:

Job Seeker stories

1. As a job seeker, I want to upload and update my resume so that recruiters can view my latest qualifications.
2. As a job seeker, I want to track the status of each application (Applied, Interview, Offered, Rejected) so that I can stay organized throughout my job search.
3. As a job seeker, I want to receive notifications when a job I'm interested in is about to close so that I don't miss application deadlines.
4. As a job seeker, I want to view a dashboard summarizing all my past and current applications so that I can quickly assess my progress.
5. As a job seeker, I want to receive AI-generated feedback on my resume so that I can improve it before applying.

Recruiter stories

6. As a recruiter, I want to create and publish job postings so that job seekers can discover and apply to open roles.
7. As a recruiter, I want to edit or close a job posting so that I can keep listings accurate and up to date.
8. As a recruiter, I want to view a list of all applicants for a specific job posting so that I can review candidates efficiently.
9. As a recruiter, I want to update an applicant's status (e.g., move them to Interview or Rejected) so that job seekers are kept informed of their progress.
10. As a recruiter, I want to search and filter candidate profiles by skills or experience so that I can identify strong matches for my open roles.

The AI also added a note advising the team to label which stories came from AI versus the team, and to check for overlap between the AI batch and the team's own stories.

Appendix B: Task 1.2 Prompts and Key Responses

Excerpts only. The complete conversation is available at the chat link in the Task 1.2 entry. Code formatting was restored for readability; the content is unchanged. Emoji in printed messages were omitted for PDF rendering.

Prompt 1

okay lets test it my code first and if all is good, pls show me the steps to safely merge it to main. Here is all of my code:

```
base.py:
from sqlalchemy.orm import declarative_base
Base = declarative_base()

connection.py:
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")
if DATABASE_URL and DATABASE_URL.startswith("postgres://"):
    DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    """ Creates a temporary database session for a single API request,
    and automatically closes it when the request is done. """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

__init__.py:
# When we use alembic, instead of it trying to find the models in the
# database folder, it will look for them in this file.
from App.models.user import User

user.py:
from sqlalchemy import Column, Integer, String, DateTime
from datetime import datetime, timezone
from App.database.base import Base

class User(Base):
    """ The User model defines the 'users' table structure in the
    PostgreSQL database. It stores credentials required for user
    registration and login. """
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True, nullable=False)
    email = Column(String(100), unique=True, index=True, nullable=False)
    # stored as a HASHED password, never plaintext
    hashed_password = Column(String(255), nullable=False)
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))
```

Prompt 2

show me step by step

AI response 1 (excerpt): test script and merge plan

The AI proposed a temporary test file in the project root and described the merge steps.

```
import sys
import os
sys.path.append(os.path.abspath(os.path.dirname(__file__)))
from app.database.connection import engine
from app.database.base import Base
from app.models.user import User

print("Attempting to connect to your Neon Cloud Database...")
try:
    Base.metadata.create_all(bind=engine)
```

```
print("Success! Your 'users' table has been created in your cloud database.")
except Exception as e:
    print("Connection Failed! Error details below:")
    print(e)
# run with: python test_db.py
```

Result 1 (terminal output pasted by the team member)

```
ImportError: cannot import name 'engine' from 'App.database.connection'
(/Users/Safadi/SOEN341-2026/App/database/connection.py)
```

AI response 2 (excerpt)

The AI attributed the error to a capitalization mismatch (App vs app), gave a corrected script using from App... imports, and asked the team member to confirm connection.py was saved and contains engine = create_engine(DATABASE_URL).

Result 2 (terminal output pasted by the team member, abbreviated)

```
File ".../App/database/connection.py", line 17, in <module>
    engine = create_engine(DATABASE_URL)
...
File ".../sqlalchemy/dialects/postgresql/psycopg.py", line 497, in import_dbapi
    import psycopg
ModuleNotFoundError: No module named 'psycopg'
```

AI response 3 (excerpt)

The AI explained that SQLAlchemy 2.0 defaults to the psycopg (v3) driver for postgresql:// URLs and recommended:

```
pip install "psycopg[binary]"
pip freeze > requirements.txt
python test_db.py
```

Result 3

Team member: "it worked"

AI response 4 (excerpt): merge steps

```
rm test_db.py
git status # confirm .env and .venv/ are NOT listed
git add App/ requirements.txt .gitignore
git commit -m "Initialize backend layout, configure SQLAlchemy, and deploy 3NF User model"
git push origin feature/setup-database
# then on GitHub: Compare & pull request -> Create pull request -> team review -> Merge
```

Appendix C: Task 1.3 Prompt and Response

Prompt (team's Sprint 1 planning notes, as submitted)

```
sprint 1 plan

init stuff (order matters ig)
- github repo
- readme
- desc of project
- tech we using
- who's on team + roles
- folder for ai logs each person
- meeting notes file

planning section
workflow n branching - gonna do github flow i think, main branch always works, ppl branch off for
features like feature/registration or whatever. PRs need 1 approval before merging. reviewer should
check it actually does what the issue says, doesnt look messy, doesnt break stuff. story is "ready"
when it has criteria + tasks + someone assigned. its "done" when merged + tested + issue closed

risks - idk some stuff that could go wrong
- team doesnt really know fastapi/react well, could slow us down, gonna give ourselves time to
  learn + pair people up
- work might not be spread evenly, some ppl doing more than others, use issues/labels to keep track
- might try to add too much stuff and not finish, so just stick to the 2 required features for now
- merge conflicts prob gonna happen, just commit small and often

effort estimation - we'll use story points i guess, fibonacci (1,2,3,5,8), talk about it as a team
when we assign. 1 = super easy like a static page, 5 = big thing like auth that touches db+api+ui.
gonna size all 15 stories once we have them all written

team capacity - 6 people, everyone doing like 4 hrs a week, sprint is 2 weeks so thats like [need
to calc this]. gonna set aside ~20% of that for ramp up time since ppl dont know the stack yet,
rest goes to the 2 features + docs

user stories - need min 15, using github issues, tasks assigned to ppl. 10 come from asking ai,
other 5+ we make ourselves and keep separate

demo - gotta show 2 features working
- registration
- login

here is my sprint planning make me a document in a proper format
```

AI response

The AI produced a formatted Sprint 1 Plan document (separate file) and replied:

I cleaned up the structure and did the capacity math for you: 6 people x 4 hrs/week x 2 weeks = 48 total hours, with ~10 hours set aside for ramp-up and ~38 left for the actual work.

Appendix D: Task 1.4 Prompts and Response

Inputs uploaded with the first prompt: the SOEN 341 Sprint 1 assignment brief (PDF) and the team member's raw AI notes (PDF) containing the user stories chat, the database testing chat, and the sprint planning notes.

Prompt 1

here is my ai doc i need you to put it all into the proper format and give it to me in a pdf

AI response 1 (summary)

The AI produced a PDF titled AI Usage Log (Sprint 1) with a cover block, a summary table, entries for Tasks 1.1 to 1.3 in the required format, and Appendices A to C holding the full prompts and key responses. It told the team member which fields were drafted and highlighted for confirmation, and flagged four things to check before submitting: (1) the Task 1.1 prompt included five existing stories rather than a clean request for 10; (2) the AI tool names and the team member's full name needed confirming; (3) the Task 1.2 reflection notes that the first AI fix did not solve the real error; (4) the code in the appendix had its formatting restored and emoji removed for PDF rendering.

Prompt 2

add this convo too thanks

AI response 2 (summary)

The AI added this conversation as Task 1.4 with this appendix, updated the summary table and entry count, and regenerated the PDF.

Appendix E: Task 1.5 Prompts and Key Responses

Excerpts and summaries only. The complete conversation is available at the chat link in the Task 1.5 entry:

<https://claude.ai/share/74720ca5-a9c7-4db0-8244-e7ebe5e4a248>. Local file paths are kept as they appeared. The JWT secret key the AI generated is redacted here and must not be reused outside local development.

Part 1: Registration fails with a 500 error

Prompt 1: whats wrong why can i register a account (with the server traceback pasted; the key line is below)

```
psycopg.errors.UndefinedColumn: column users.full_name does not exist
File ".../App/routers/auth.py", line 21, in register
    if db.query(User).filter(User.email == email).first():
```

AI response 1 (summary): The Python User model and the Postgres users table do not match. The table was created before full_name was added, and create_all() never adds columns to an existing table. It offered three fixes: add the column, drop and recreate the table, or use Alembic migrations.

Prompt 2: the team member pasted the User model (with full_name, email, hashed_password, role, created_at). **AI response 2 (summary):** confirmed the model was correct and the database was out of date; repeated the fixes and told the team member to compare the table columns with the model.

Prompt 3: huh why are you using sql language im only using sqlalchemy orm with python to talk to postgresql which is a neon database

AI response 3 (summary): apologised for the confusion and gave a Python-only fix, a script placed in the project root:

```
from App.database.base import Base
from App.database.session import engine # guessed file name, wrong for this project
from App.models.user import User

Base.metadata.drop_all(bind=engine)
Base.metadata.create_all(bind=engine)
```

Prompts 4 and 5: why do i need to delete it / where do i add this delete file? **AI responses (summary):** create_all() only checks whether a table exists and does not compare columns, so the table must be recreated (or altered by hand). The script belongs in the project root, next to the App folder, and is run from there with the virtual environment on.

Part 2: Fixing the reset script's import

Result (terminal output pasted by the team member):

```
ModuleNotFoundError: No module named 'App.database.session'
```

AI response (summary): the guessed file name was wrong; find the file that creates the engine (grep -rn "create_engine" App) and use its name in the import. The team member pasted the database file (the engine and get_db code), the AI confirmed it was the right one, and the team member updated the import to match the project's real file name. **Prompts:** so i connect it to the file of the code i just provided? / okay done now what? / how to restart the app? by just running it again? / can i delete the reset_db file.

AI responses (summary): run the script, restart the backend with Ctrl + C then uvicorn, and delete the script afterwards (or add it to .gitignore) so nobody wipes shared data by accident.

Part 3: Registration fails with a 400 on OPTIONS (CORS)

Prompt: the team member pasted the log line below.

```
INFO: 127.0.0.1:55946 - "OPTIONS /api/v1/auth/register HTTP/1.1" 400 Bad Request
```

AI response (summary): explained the browser's CORS permission check and asked to see the CORS setup. The team member pasted main.py and then config.py. The AI found that the code was correct and that the allowed origins list only contained the default http://localhost:5173. The team member reported the frontend was running at http://localhost:5175, so the AI suggested this line in .env, then restarting the backend:

```
CORS_ORIGINS=http://localhost:5173,http://localhost:5174,http://localhost:5175
```

Prompt: okay lets run everything again. **AI response (summary):** stop old servers, start the backend and frontend in two terminals, and look for OPTIONS 200 then POST 201 in the backend log. Registration then succeeded (201 Created).

Part 4: Login fails with a 500 (missing JWT secret key)

Prompt: okay i finally registered but login doesnt work now, followed by the backend log:

```
POST /api/v1/auth/login HTTP/1.1" 500 Internal Server Error
File ".../App/core/security.py", line 41, in create_access_token
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
TypeError: Expected a string or bytes value
```

AI response (summary): login found the account and accepted the password, then crashed when signing the token because SECRET_KEY was empty. It guessed the environment variable names might not match. **Prompts:** so from who do i get this secret key / so it can be anything or does someone make it up / okay make me it to add to .env. **AI responses (summary):** the key is a self-made random string used to sign tokens; it ran a command to generate one and gave this line for .env:

```
JWT_SECRET_KEY=[REDACTED - 64-character random hex string, generated for local development]
```

Prompt: before i start for ref this is security.py (the file was not attached; the team member then pasted it). **AI response (summary):** security.py reads JWT_SECRET_KEY, so the variable name was already correct and the key was simply missing from .env; no code change was needed. Restart the backend and try login again.

Prompts: so the code and everything was fine the only thing that we did was on my own local machine right? / how did we fix the registration part again? **AI responses (summary):** the fixes were local .env values plus a reset of the Neon tables (which affects teammates if they share one database); the code itself did not change. Suggested a .env.example file for the team. Recapped the two registration problems (outdated table, CORS) and the separate login problem (missing secret key).

Part 5: README review

Prompt: the team member uploaded the SOEN 341 Sprint 1 assignment brief, pasted the README (title, technologies, team, setup steps, run steps), and asked: is this readme complete?

AI response (summary): not yet. Missing from the required README list: identified problem, proposed solution, proposed features, and the GitHub repository link. Also noted a duplicated Krish line, no git clone step, .env setup that only covered the database URL, no folder named for the frontend commands, Alembic listed while main.py still used the temporary create_all(), and CareerNet vs CareerConnect naming. Provided draft Problem, Solution, Features, and Repository sections and a replacement Step 5 for the .env variables, and noted the generative AI feature and original feature still need the team's own decisions.

Part 6: Adding this conversation to the AI log (first instruction)

Final prompt (as submitted, with the existing AI Usage Log PDF uploaded):

```
include this chat into document please and include the fact that i told you to include this chat
https://claude.ai/share/74720ca5-a9c7-4db0-8244-e7ebe5e4a248
```

AI response (summary): The AI added this conversation as Task 1.5 with this appendix, updated the summary table and the entry count from 4 to 5, recorded in the Task 1.5 entry and in this appendix that the team member instructed it to add the chat, redacted the secret key, and regenerated the PDF. It highlighted the validation, decision, and reflection wording as draft for the team member to confirm, including the result of the final login test, which the conversation did not confirm.

Part 7: Features and GitHub issues

Prompt (summary): the team member said the features are the user stories, which they had already turned into GitHub issues, and pasted the repository issue list (21 open, 2 closed). The list included these user stories: #5 Job Seeker Account Registration, #6 Job Seeker Login, #16 to #25 (the ten stories from Appendix A, in the same order), and #26 to #28 (career goal and CV-based job matching, job search and filtering, save jobs to favourites). It also included Sprint 1 task issues #7, #8, and #10 to #13.

AI response (summary): gave a README Features section that mirrors the issues (grouped into job seeker and recruiter features, each with its issue number), a Repository section with <https://github.com/Sayf-01/SOEN341-2026>, and noted that #20 (AI-generated resume feedback) is the generative AI feature while the additional original feature is still a team decision. It then flagged three problems: (1) #10 and #12 are both titled "Build login backend endpoint" and #11 and #13 are both titled "Build login UI form", so the extras should be closed and the tasks attached to their parent stories; (2) AI and team stories are not labeled, so labels such as ai-generated and team-generated should be added, and the labels sprint-1 and Sprint-2 differ in capitalization; (3) story #26 was labeled "AI suggestion 3" in the Task 1.1 prompt, so its source should be checked before it is presented as team-generated. It also noted the assignment expects extra original team stories beyond the AI's.

Part 8: Adding the later messages to the AI log (second instruction)

Prompt (as submitted):

```
add this to the ai log document too
```

AI response (summary): The AI extended Task 1.5 and this appendix with Part 7 (the features and issue-list discussion) and this Part 8, updated the Task 1.5 title, purpose, suggested content, validation, and decision text, and regenerated the PDF. The entry count stays at 5 because these messages belong to the same conversation as the rest of Task 1.5. The team member gave this instruction; the AI added the entry.